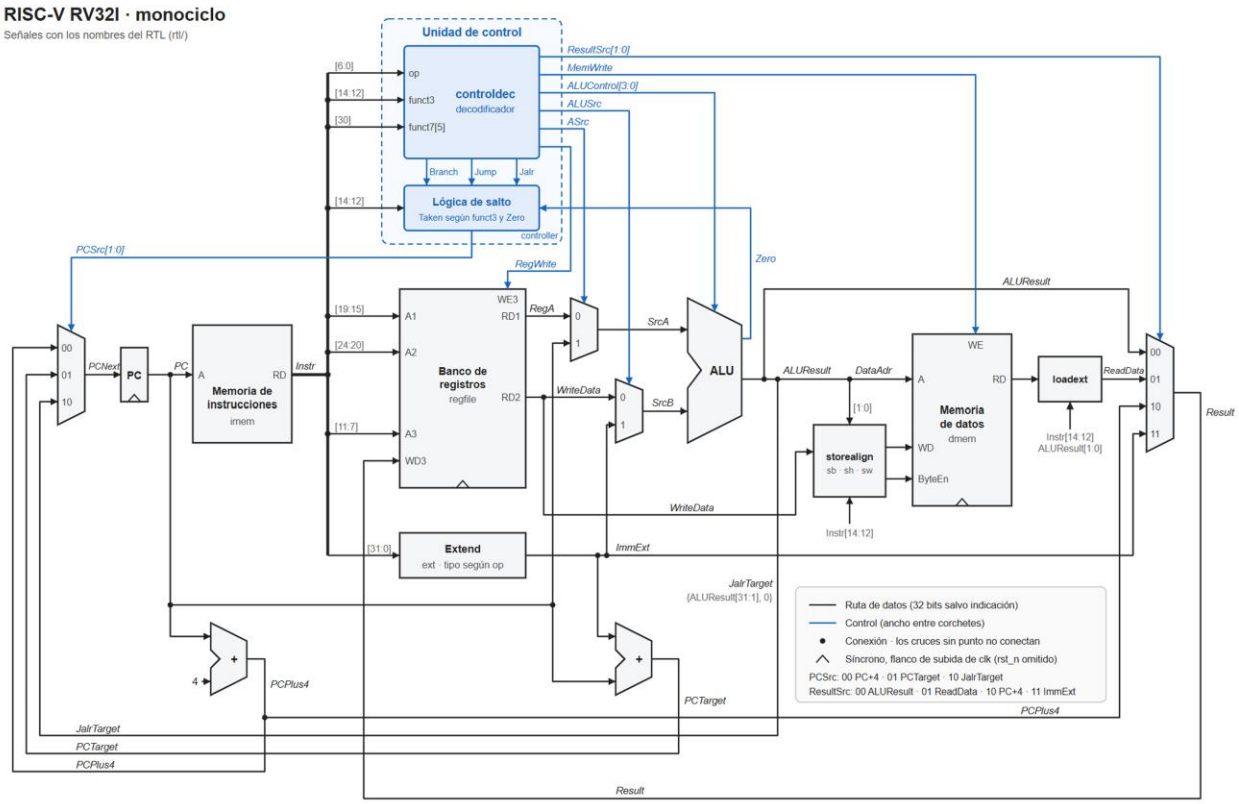
Unidad de control del RISC-V RV32I monociclo

Antonio Rodríguez



1. Introducción

En este documento se intenta explicar el funcionamiento de la unidad de control de la implementación específica del microprocesador RISC-V monociclo que hemos hecho nosotros y que puede diferir ligeramente de otras implementaciones que te puedas encontrar.

La unidad de control (bloque `controller` del RTL) traduce cada instrucción en un conjunto de señales que configuran la ruta de datos durante un único ciclo de reloj. Es un circuito puramente combinacional: no tiene estado ni flanco de reloj, y sus salidas se estabilizan poco después de que la memoria de instrucciones entrega `Instr`.

Está formada por dos bloques:

- **controldec** (decodificador): mira el código de operación y los campos de función y genera las señales que gobiernan la ALU, los multiplexores de operandos, la escritura en registros y memoria, y la selección del resultado.
- **Lógica de salto**: combina las indicaciones de salto del decodificador con el campo `funct3` y el indicador `Zero` de la ALU para decidir de dónde se toma el siguiente PC (`PCSrc[1:0]`).

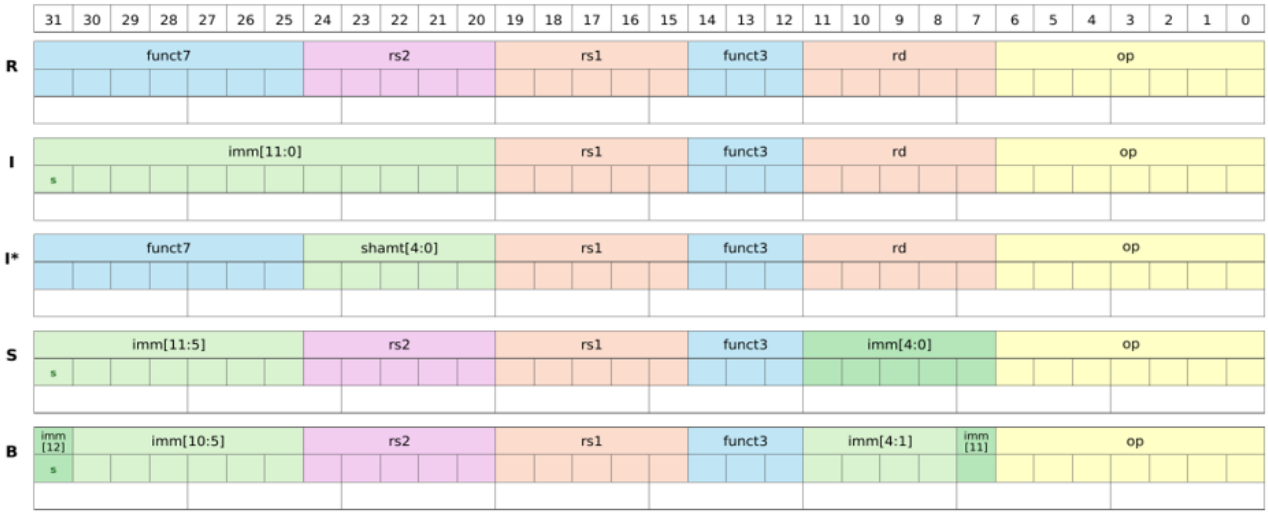
En un procesador monociclo cada instrucción recorre todo el camino en un ciclo: búsqueda en `imem`, lectura de registros, cálculo en la ALU, acceso a `dmem` y escritura del resultado. Solo tres elementos son síncronos (marcados con el triángulo de reloj en el esquema): el registro **PC**, el **banco de registros** y la **memoria de datos**. Las señales de control deciden qué se escribe en ellos en el flanco de subida que cierra el ciclo.

2. Entradas de la unidad de control

La unidad de control recibe cuatro campos extraídos directamente de la instrucción y una señal de estado de la ALU.

Entrada	Bits de `Instr`	Destino	Para qué sirve
`op`	[6:0]	controldec	Identifica el formato y la clase de instrucción (R, I, carga, almacenamiento, salto condicional, `jal`, `jalr`, `lui`, `auipc`).
`funct3`	[14:12]	controldec	Distingue la operación dentro de una clase: `add`/`xor`/`slt`…, o el tipo de comparación en los saltos.
`funct7[5]`	[30]	controldec	Único bit de `funct7` que RV32I necesita: separa `add`/`sub` y `srl`/`sra`.
`funct3`	[14:12]	Lógica de salto	Indica qué condición de salto evaluar (`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`).
`Zero`	— (sale de la ALU)	Lógica de salto	Vale 1 cuando el resultado de la ALU es cero. Es la única información de la ALU que vuelve a la unidad de control.

Observa que `funct3` llega por dos ramas distintas: una al decodificador y otra a la lógica de salto. Basta con el bit 30 de `funct7` porque en RV32I ningún otro bit de ese campo cambia la operación.



3. El decodificador controldec

El decodificador genera nueve señales a partir de `op`, `funct3` y `funct7[5]`. Seis van directamente a la ruta de datos y tres son internas a la unidad de control.

Hacia la ruta de datos:

- `RegWrite` → habilitación de escritura `WE3` del banco de registros.
- `ALUSrc` → multiplexor del segundo operando (`SrcB`).
- `ASrc` → multiplexor del primer operando (`SrcA`).
- `ALUControl[3:0]` → operación de la ALU.
- `MemWrite` → habilitación de escritura `WE` de la memoria de datos.
- `ResultSrc[1:0]` → multiplexor final que elige el valor `Result` escrito en el registro destino.

Hacia la lógica de salto:

- `Branch` → la instrucción es un salto condicional (opcode `1100011`).
- `Jump` → la instrucción es `jal` (salto incondicional relativo al PC).
- `Jalr` → la instrucción es `jalr` (salto indirecto a través de un registro).

Conceptualmente el decodificador hace dos cosas. Primero, una **decodificación principal** basada solo en `op`, que fija `RegWrite`, `ALUSrc`, `ASrc`, `MemWrite`, `ResultSrc`, `Branch`, `Jump` y `Jalr`. Segundo, una **decodificación de la ALU** que usa además `funct3` y `funct7[5]` para elegir el valor concreto de `ALUControl`.

Un rasgo particular de este diseño es que **no existe señal `ImmSrc`**. En el diseño clásico de Harris y Harris el control indica al extensor qué formato de inmediato usar; aquí el bloque `Extend` recibe la instrucción completa ([31:0]) y decide el formato por sí mismo según `op` ("ext · tipo según op"). Esa parte de la decodificación está, por tanto, desplazada fuera de la unidad de control.

Cómo está escrito en `controldec.sv`. El módulo recibe la instrucción completa, pero solo lee `instr[6:0]` (op), `instr[14:12]` (funct3) e `instr[30]` (funct7[5]). Todo se resuelve en un único `always_comb` con tres rasgos destacables:

- **Valores por defecto**: antes del `case` todas las salidas se ponen a 0. Cada rama solo activa lo que su instrucción necesita.
- **Tipo R y tipo I aritmética comparten rama**, porque usan la misma decodificación de `funct3`. Las distingue el bit 5 del opcode (1 en `0110011`, 0 en `0010011`): `ALUSrc = ~instr[5]` y la resta solo se elige si `instr[5] & instr[30]`, con lo que un `addi` nunca se confunde con `sub`.
- **En los saltos solo se mira `funct3[2:1]`** para elegir la operación de la ALU. El bit `funct3[0]`, que separa cada salto de su negado (`beq`/`bne`, `blt`/`bge`…), lo usa después la lógica de salto.

4. Lógica de salto y generación de PCSrc

La lógica de salto decide cuál de las tres direcciones candidatas se carga en el PC al final del ciclo. Lo hace mediante `PCSrc[1:0]`, que gobierna el multiplexor de tres entradas situado antes del registro PC.

PCSrc	Siguiente PC	Cuándo

PCSrc	Siguiente PC	Cuándo
00	PCPlus4 = PC + 4	Ejecución secuencial, o salto condicional no tomado.
01	PCTarget = PC + ImmExt	jal, o salto condicional tomado.
10	JalrTarget = {ALUResult[31:1], 0}	jalr.

La decisión se puede expresar así:

1. Si Jalr = 1 → PCSrc = 10.
2. Si no, si Jump = 1, o Branch = 1 y la condición se cumple (Taken = 1) → PCSrc = 01.
3. En cualquier otro caso → PCSrc = 00.

Cómo se evalúa la condición con un único bit Zero. La ALU solo devuelve Zero, no indicadores de signo ni de acarreo. Por eso el decodificador debe elegir en ALUControl la operación que convierte cada comparación en una prueba de "resultado igual a cero", y la lógica de salto interpreta Zero según funct3:

Instrucción	funct3	Operación en la ALU	Salto tomado si
beq	000	resta (rs1 − rs2)	Zero = 1
bne	001	resta (rs1 − rs2)	Zero = 0
blt	100	slt (rs1 < rs2 con signo)	Zero = 0 (resultado 1)
bge	101	slt	Zero = 1 (resultado 0)
bltu	110	sltu (sin signo)	Zero = 0
bgeu	111	sltu	Zero = 1

En controller.sv la condición se escribe como un case sobre funct3, línea a línea como en la tabla: Zero para beq, bge y bgeu; ~Zero para bne, blt y bltu; y 0 para los códigos 010 y 011, que no existen. Para los seis saltos válidos equivale a la forma compacta Taken = Zero XOR funct3[0] XOR funct3[2]. La selección del PC es una sola línea: PCSrc = Jalr ? 2'b10 : {1'b0, Jump | (Branch & Taken)}.

Las dos direcciones de destino se calculan fuera de la unidad de control. PCTarget sale de un sumador dedicado (PC + ImmExt), de modo que la ALU queda libre para la comparación en el mismo ciclo. JalrTarget reutiliza la ALU, que calcula rs1 + inmediato, y fuerza a 0 el bit menos significativo, como exige la especificación de jalr.

5. Las líneas de control, una a una

RegWrite (1 bit)

Habilita la escritura en el banco de registros (entrada WE3). Cuando vale 1, en el flanco de subida se escribe Result (entrada WD3) en el registro indicado por A3 = Instr[11:7] (rd). Vale 1 en todas las instrucciones que producen un resultado: tipo R, tipo I aritméticas, cargas, jal, jalr, lui y auipc. Vale

0 en almacenamientos y saltos condicionales, que no tienen registro destino. Nunca debe quedar indeterminado: un valor espurio corrompería un registro.

ALUSrc (1 bit)

Selecciona el segundo operando de la ALU (`SrcB`).

- 0 → `WriteData` = `RD2`, el contenido de rs2. Se usa en las instrucciones tipo R y en los saltos condicionales, que comparan dos registros.
- 1 → `ImmExt`, el inmediato extendido. Se usa en tipo I aritméticas, cargas, almacenamientos (cálculo de dirección), `jalr` y `auipc`.

Aunque en los almacenamientos `SrcB` es el inmediato, `RD2` sigue llegando por su propio camino (`WriteData`) hasta `storealign`: es el dato que se escribirá en memoria.

ASrc (1 bit)

Selecciona el primer operando de la ALU (`SrcA`). Es una señal añadida respecto al diseño de libro de texto.

- 0 → `RegA` = `RD1`, el contenido de rs1. Es el caso normal.
- 1 → el valor del `PC`. Solo lo necesita `auipc`, que calcula PC + (inmediato << 12) en la ALU y lo escribe en rd por la entrada 00 del multiplexor de resultado.

ALUControl[3:0] (4 bits)

Indica a la ALU qué operación realizar. Con 4 bits caben hasta 16 operaciones; RV32I necesita diez: suma, resta, AND, OR, XOR, desplazamiento lógico a la izquierda, desplazamiento lógico y aritmético a la derecha, `slt` y `sltu`. La codificación es la del módulo alu.sv y se detalla en las dos tablas que siguen a esta lista. El valor se elige así:

- **Cargas, almacenamientos, `jalr` y `auipc`**: siempre suma (dirección o destino = base + desplazamiento).
- **Saltos condicionales**: resta para `beq`/`bne`, `slt` para `blt`/`bge`, `sltu` para `bltu`/`bgeu` (sección 4).
- **Tipo R**: según `funct3`; `funct7[5]` distingue `add`/`sub` (funct3 = 000) y `srl`/`sra` (funct3 = 101).
- **Tipo I aritméticas**: según `funct3`; `funct7[5]` solo se mira en `srli`/`srai`. En `addi` el bit 30 forma parte del inmediato, de modo que si se tuviera en cuenta, un `addi` con inmediato negativo se ejecutaría como resta por error.
- **`jal` y `lui`**: la ALU no interviene en el resultado; el valor es indiferente.

Tabla de verdad de la ALU

Codificación tomada de `alu.sv`. Entradas: `SrcA` (puerto `a`), `SrcB` (puerto `b`) y `ALUControl[3:0]`. Salidas: `ALUResult` (puerto `result`) y `Zero`. Las comparaciones y los desplazamientos se calculan en dos submódulos, `alu_cmp` y `alu_shifter`, y el `case` principal solo elige qué resultado sale.

ALUControl	Operación	ALUResult	Calculado en	Instrucciones que la usan
0000	Suma	SrcA + SrcB	alu	add, addi, cargas,

ALUControl	Operación	ALUResult	Calculado en	Instrucciones que la usan
				almacenamientos, jalr, auipc
0001	Resta	SrcA − SrcB	alu	sub, beq, bne
0010	AND	SrcA & SrcB	alu	and, andi
0011	OR	SrcA | SrcB	alu	or, ori
0101	Menor que, con signo	1 si SrcA < SrcB (con signo), si no 0	alu_cmp	slt, slti, blt, bge
1000	XOR	SrcA ^ SrcB	alu	xor, xori
1001	Desplazamiento lógico a la izquierda	SrcA << SrcB[4:0]	alu_shifter	sll, slli
1010	Desplazamiento lógico a la derecha	SrcA >> SrcB[4:0] (entran ceros)	alu_shifter	srl, srli
1011	Desplazamiento aritmético a la derecha	SrcA >>> SrcB[4:0] (entra el signo)	alu_shifter	sra, srai
1100	Menor que, sin signo	1 si SrcA < SrcB (sin signo), si no 0	alu_cmp	sltu, sltiu, bltu, bgeu
0100, 0110, 0111, 1101, 1110, 1111	No usados	0xDEADBEEF	alu	—

En todos los casos `Zero` = 1 si `ALUResult` = 0 (`assign zero = (result == 32'h0)`); con un código no usado vale 0. Los desplazamientos solo usan los 5 bits bajos de `SrcB`, y `slt`/`sltu` devuelven 0x00000001 o 0x00000000. El valor 0xDEADBEEF es una ayuda de depuración: si aparece en un registro, controldec ha generado un código no previsto.

Cómo genera controldec el valor de ALUControl

Esta codificación no coincide con los campos de la instrucción, así que el decodificador tiene que traducir `funct3` (y `funct7[5]` donde importa) con una tabla:

funct3	Tipo R (op 0110011)	Tipo I aritmética (op 0010011)	Salto condicional (op 1100011)
000	add → 0000 · sub → 0001 (según funct7[5])	addi → 0000 (funct7[5] no se mira)	beq → 0001

funct3	Tipo R (op 0110011)	Tipo I aritmética (op 0010011)	Salto condicional (op 1100011)
001	sll → 1001	slli → 1001	bne → 0001
010	slt → 0101	slti → 0101	—
011	sltu → 1100	sltiu → 1100	—
100	xor → 1000	xori → 1000	blt → 0101
101	srl → 1010 · sra → 1011 (según funct7[5])	srli → 1010 · srai → 1011 (según funct7[5])	bge → 0101
110	or → 0011	ori → 0011	bltu → 1100
111	and → 0010	andi → 0010	bgeu → 1100

El resto de instrucciones no dependen de `funct3`: cargas, almacenamientos, `jalr` y `auipc` usan siempre 0000 (suma). En `jal` y `lui` la rama del `case` no asigna `ALUControl`, así que se queda en su valor por defecto 0000: la ALU suma rs1 + rs2 con lo que haya en esos campos, pero el resultado no se usa porque `ResultSrc` elige `PCPlus4` o `ImmExt`. Los saltos con `funct3` = 01x, que no existen, usan la resta como valor por defecto.

MemWrite (1 bit)

Habilita la escritura en la memoria de datos (entrada `WE`). Vale 1 solo en `sb`, `sh` y `sw`. La escritura ocurre en el flanco de subida, en la dirección `DataAdr` = `ALUResult`, con el dato y la máscara de bytes que prepara `storealign`. En el resto de instrucciones debe valer 0 para no alterar la memoria. La lectura de `dmem` es combinacional y no necesita señal de habilitación.

ResultSrc[1:0] (2 bits)

Selecciona qué valor se escribe en el registro destino a través de `Result`.

ResultSrc	Valor de Result	Instrucciones
00	ALUResult	Tipo R, tipo I aritméticas, auipc
01	ReadData (salida de loadext)	lb, lh, lw, lbu, lhu
10	PCPlus4	jal, jalr (dirección de retorno)
11	ImmExt	lui

La entrada 11 es otra particularidad del diseño: `lui` lleva el inmediato (ya desplazado por `Extend`) directamente al registro, sin pasar por la ALU. En almacenamientos y saltos condicionales el valor es indiferente porque `RegWrite` = 0.

Branch, Jump y Jalr (1 bit cada una)

Son señales internas: viajan del decodificador a la lógica de salto y no tocan la ruta de datos. Cada una se activa con un único opcode: `Branch` con `1100011`, `Jump` con `jal` (`1101111`) y `Jalr` con `jalr` (`1100111`). Como mucho una de las tres está a 1 en cada instrucción.

PCSrc[1:0] (2 bits)

Salida de la lógica de salto que controla el multiplexor del siguiente PC (`PCNext`): 00 = PC + 4, 01 = `PCTarget`, 10 = `JalrTarget`. Su generación está detallada en la sección 4.

Zero (1 bit, señal de estado)

No es una señal de control sino de estado: va en sentido contrario, de la ALU a la lógica de salto. Vale 1 cuando `ALUResult` = 0 y solo se tiene en cuenta cuando `Branch` = 1.

6. Control implícito: bloques que leen la instrucción directamente

Tres bloques de la ruta de datos no reciben señales de la unidad de control, pero sí toman campos de `Instr` y actúan como pequeños decodificadores locales. Para entender el control completo hay que tenerlos en cuenta.

Extend

Recibe `Instr[31:0]` y produce `ImmExt` de 32 bits. Según `op` reconoce el formato del inmediato y reordena y extiende sus bits:

- **I** (tipo I aritméticas, cargas, `jalr`): `Instr[31:20]` con extensión de signo.
- **S** (almacenamientos): `Instr[31:25]` y `Instr[11:7]` concatenados, con signo.
- **B** (saltos condicionales): desplazamiento en múltiplos de 2, con signo.
- **J** (`jal`): desplazamiento de 21 bits en múltiplos de 2, con signo.
- **U** (`lui`, `auipc`): `Instr[31:12]` seguido de 12 ceros.

Para `slli`/`srli`/`srai` solo importan los 5 bits bajos del inmediato (`shamt`), que la ALU usa como cantidad de desplazamiento.

La figura muestra los formatos de instrucción con los bits del inmediato que contiene cada campo; es la correspondencia que deshace `Extend`. La «s» marca el bit de signo, y la fila I* es la variante de tipo I para `slli`, `srli` y `srai`.



storealign

Prepara las escrituras de tamaño byte o media palabra. Recibe el dato (`WriteData` = rs2), los dos bits bajos de la dirección (`DataAdr[1:0]`) y `funct3` (`Instr[14:12]`). Genera:

- `WD`: el dato replicado o desplazado a la posición correcta dentro de la palabra de 32 bits.
- `ByteEn`: máscara de 4 bits que indica qué bytes de la palabra se escriben.

Instrucción	`funct3`	`ByteEn` según `DataAdr[1:0]`
`sb`	000	Un byte: 0001, 0010, 0100 o 1000
`sh`	001	Media palabra: 0011 (dirección 00) o 1100 (dirección 10)
`sw`	010	Palabra completa: 1111

La escritura solo tiene efecto si además `MemWrite` = 1: `ByteEn` dice *qué* bytes, `MemWrite` dice *si* se escribe.

loadext

Adapta la palabra leída de memoria al tipo de carga. Recibe la palabra de `dmem`, `funct3` (`Instr[14:12]`) y `ALUResult[1:0]`. Selecciona el byte o la media palabra direccionados y los extiende a 32 bits:

Instrucción	funct3	Resultado en ReadData
lb	000	Byte con extensión de signo
lh	001	Media palabra con extensión de signo
lw	010	Palabra completa
lbu	100	Byte con extensión de ceros
lhu	101	Media palabra con extensión de ceros

El valor de ReadData solo llega al registro destino si ResultSrc = 01 y RegWrite = 1.

7. Tabla resumen de las señales de control

Valores que genera controldec.sv para cada clase de instrucción. Las celdas marcadas con * son el valor por defecto del RTL (todas las salidas empiezan a 0) y no influyen en el resultado; en el diseño teórico serían indiferentes. RegWrite y MemWrite nunca son indiferentes.

Clase	op	RegWrite	ALUSrc	ASrc	MemWrite	ResultSrc	Operación ALU	Branch	Jump	Jalr	PCSrc
Tipo R	0110011	1	0	0	0	00	según funct3 y funct7[5]	0	0	0	00
Tipo I aritmética	0010011	1	1	0	0	00	según funct3 (funct7[5] solo en desplazamientos)	0	0	0	00
Carga	0000011	1	1	0	0	01	suma	0	0	0	00
Almacenamiento	0100011	0	1	0	1	00*	suma	0	0	0	00
Salto condicional	1100011	0	0	0	0	00*	resta / slt / sltu	1	0	0	01 si se toma, 00 si

Clase	op	RegWrite	ALUSrc	ASrc	MemWrite	ResultSrc	Operación ALU	Branch	Jump	Jalr	PCSrc
											no
`jal`	1101111	1	0*	0*	0	10	suma*	0	1	0	01
`jalr`	1100111	1	1	0	0	10	suma	0	0	1	10
`lui`	0110111	1	0*	0*	0	11	suma*	0	0	0	00
`auipc`	0010111	1	1	1	0	00	suma	0	0	0	00
Otro opcode	—	0	0	0	0	00	suma	0	0	0	00 (se comporta como `nop`)

La columna "Operación ALU" describe la función que se pide a la ALU. Sus códigos en `alu.sv` son: suma = 0000, resta = 0001, `slt` = 0101 y `sltu` = 1100; el resto está en la tabla de verdad de la sección 5.

8. Recorrido de ejemplos

`add x5, x6, x7` — Lectura de x6 y x7 por `RD1` y `RD2`. `ASrc` = 0 y `ALUSrc` = 0 llevan ambos registros a la ALU, que suma (`funct3` = 000, `funct7[5]` = 0). `ResultSrc` = 00 envía `ALUResult` a `WD3` y `RegWrite` = 1 lo escribe en x5. `PCSrc` = 00: el PC avanza 4.

`lw x5, 8(x6)` — `Extend` genera 8 (formato I). `ALUSrc` = 1, la ALU suma x6 + 8 y produce `DataAdr`. `dmem` lee la palabra, `loadext` la deja intacta (`funct3` = 010), `ResultSrc` = 01 la lleva a `Result` y `RegWrite` = 1 la escribe en x5.

`sb x7, 3(x6)` — La ALU calcula x6 + 3 (`ALUSrc` = 1). `storealign` coloca el byte bajo de x7 en la posición indicada por `DataAdr[1:0]` y genera la máscara correspondiente. `MemWrite` = 1 escribe solo ese byte; `RegWrite` = 0.

`beq x1, x2, etiqueta` — `ALUSrc` = 0 y la ALU resta x1 − x2. En paralelo, el sumador de destino calcula `PCTarget` = PC + `ImmExt`. Con `Branch` = 1 y `funct3` = 000, el salto se toma si `Zero` = 1: `PCSrc` = 01. Si no, `PCSrc` = 00. `RegWrite` y `MemWrite` valen 0.

`blt x1, x2, etiqueta` — Igual que `beq`, pero la ALU calcula `slt`: el resultado es 1 si x1 < x2. El salto se toma cuando el resultado no es cero, es decir, con `Zero` = 0.

`jal x1, función` — `Jump` = 1 fuerza `PCSrc` = 01 y el PC pasa a `PCTarget`. Al mismo tiempo `ResultSrc` = 10 y `RegWrite` = 1 guardan `PCPlus4` en x1 como dirección de retorno.

`jalr x0, 0(x1)` (retorno) — `ALUSrc` = 1, la ALU suma x1 + 0 y se forma `JalrTarget` con el bit 0 a cero. `Jalr` = 1 da `PCSrc` = 10. La escritura de `PCPlus4` en x0 no tiene efecto porque x0 vale siempre 0.

`lui x5, 0x12345` — `Extend` produce 0x12345000. `ResultSrc` = 11 lo envía directamente a `WD3` y `RegWrite` = 1 lo escribe en x5. La ALU calcula algo irrelevante.

`auipc x5, 0x1` — `ASrc` = 1 lleva el PC a `SrcA`, `ALUSrc` = 1 lleva 0x00001000 a `SrcB`, y la ALU suma. `ResultSrc` = 00 escribe PC + 0x1000 en x5.

9. Notas y supuestos

Codificación de `ALUControl`: `alu.sv` y `controldec.sv` usan los mismos códigos para las diez operaciones.

Condición de salto: confirmada en `controller.sv` (sección 4).

Inmediatos: `extend.sv` monta los cinco formatos (I, S, B, U, J) exactamente como en la sección 6; para cualquier otro opcode da 0.

Instrucciones no contempladas: `fence`, `ecall`, `ebreak` y las CSR no entran en ninguna rama del `case`. Gracias a los valores por defecto (`RegWrite`, `MemWrite`, `Branch`, `Jump` y `Jalr` a 0) se comportan como un `nop` y el PC avanza 4.

Accesos desalineados: no se detectan. `sh`/`lh` ignoran `addr[0]`, y `sw`/`lw` ignoran `addr[1:0]` porque `dmem` indexa por palabras. Un acceso desalineado lee o escribe en la posición alineada inferior sin avisar.

Tamaño de las memorias: `imem` y `dmem` tienen 64 palabras (256 bytes) cada una, pero se indexan con `addr[31:2]`. Una dirección de 256 o más cae fuera del array: en simulación la lectura da X y la escritura se pierde. Indexar con `addr[7:2]` lo dejaría acotado.

Registro x0: `regfile.sv` devuelve 0 al leer x0 e ignora las escrituras en él, así que `jalr x0, 0(x1)` funciona como retorno sin efectos laterales.